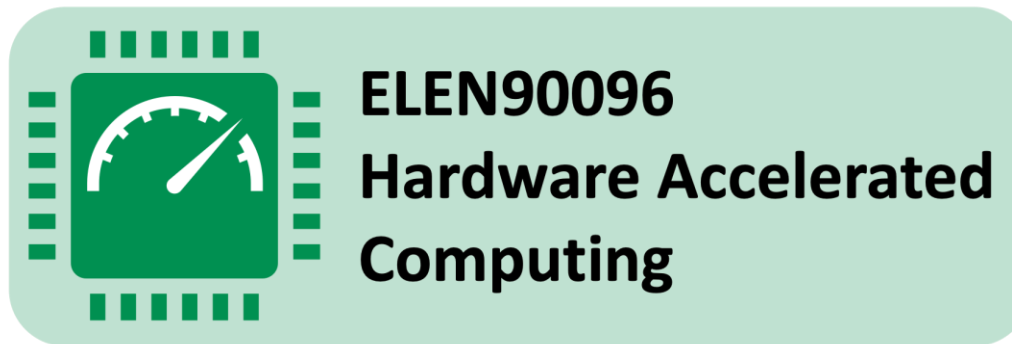
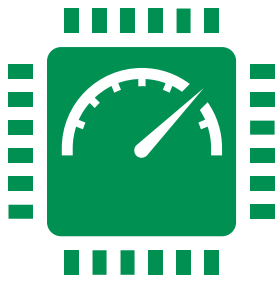


[9] CNN Optimizations



Design Project



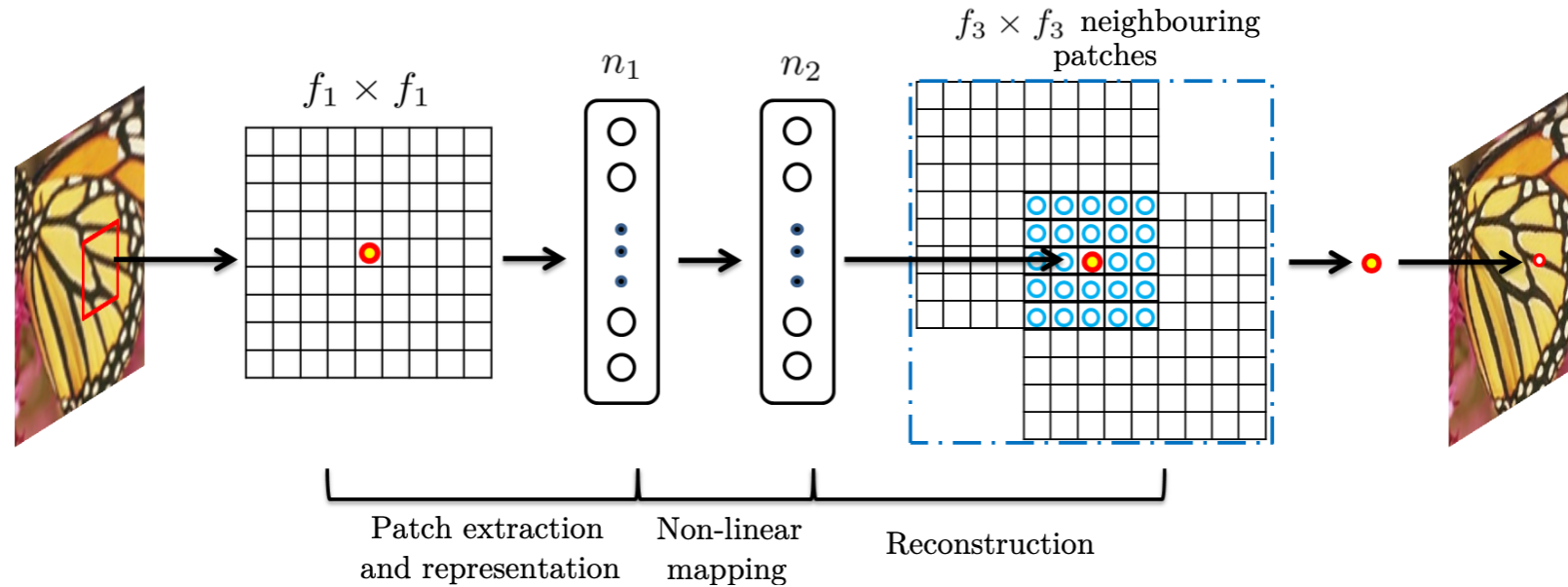
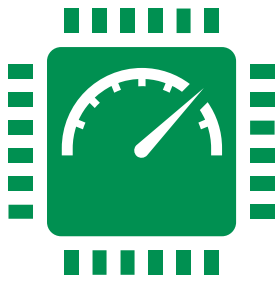
Major project milestones:

1. **Golden Reference** code submission – functionally correct, baseline C/C++ sequential implementation of SRCNN
2. **Status Update** – oral presentation to discuss approach, progress, and remaining tasks
3. **Final Report** – complete documentation of your solution and its performance, along with HLS and PYNQ host code submission

Deliverable	Due	Weight
Golden Reference	5 Oct	5%
Status Update	12 Oct	10%
Final Report	19 Oct*	25%

See Canvas assignments for additional details on what is expected for each milestone

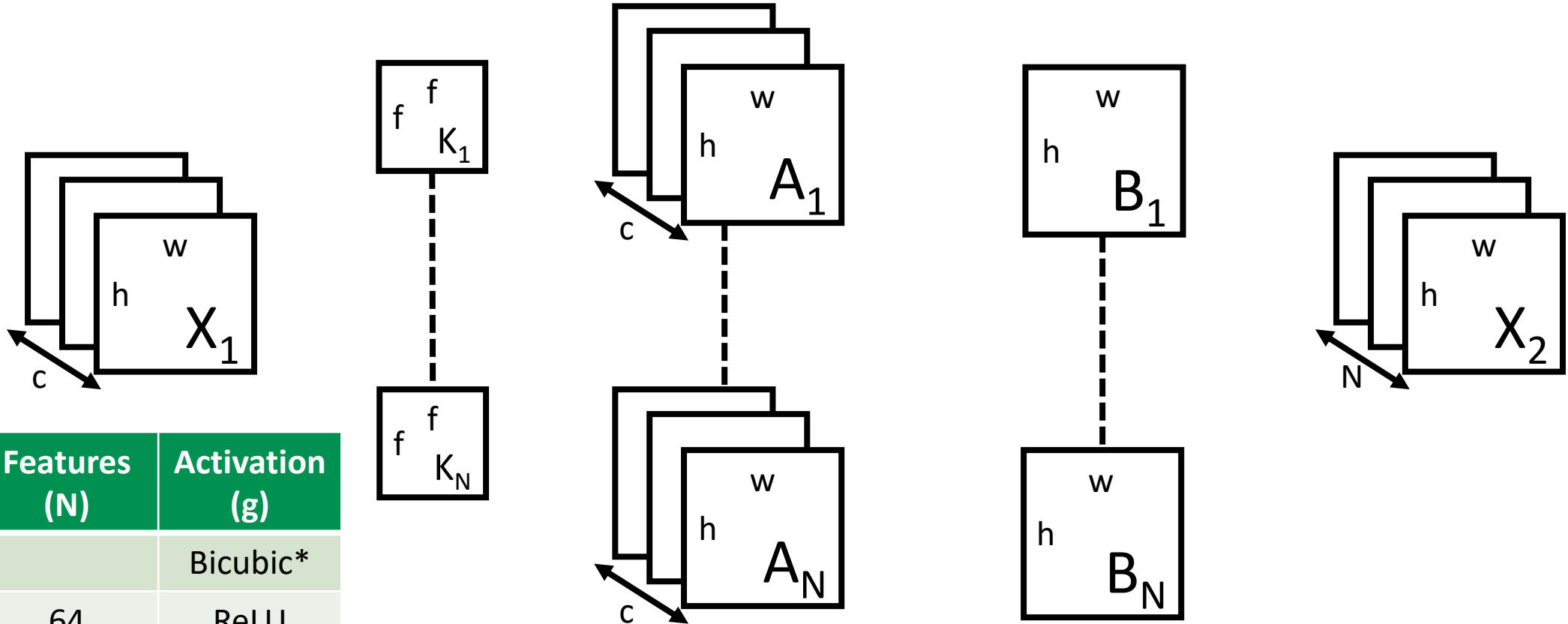
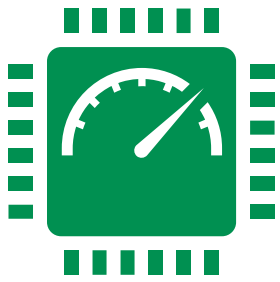
SRCNN



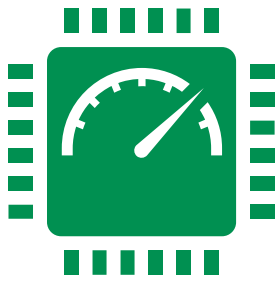
Layer	Kernel	Features	Activation
0	1 x 1	1	Bicubic*
1	9 x 9	64	ReLU
2	1 x 1	32	ReLU
3	5 x 5	1	None

Project Goal: Accelerate SRCNN on Kria KV260 board

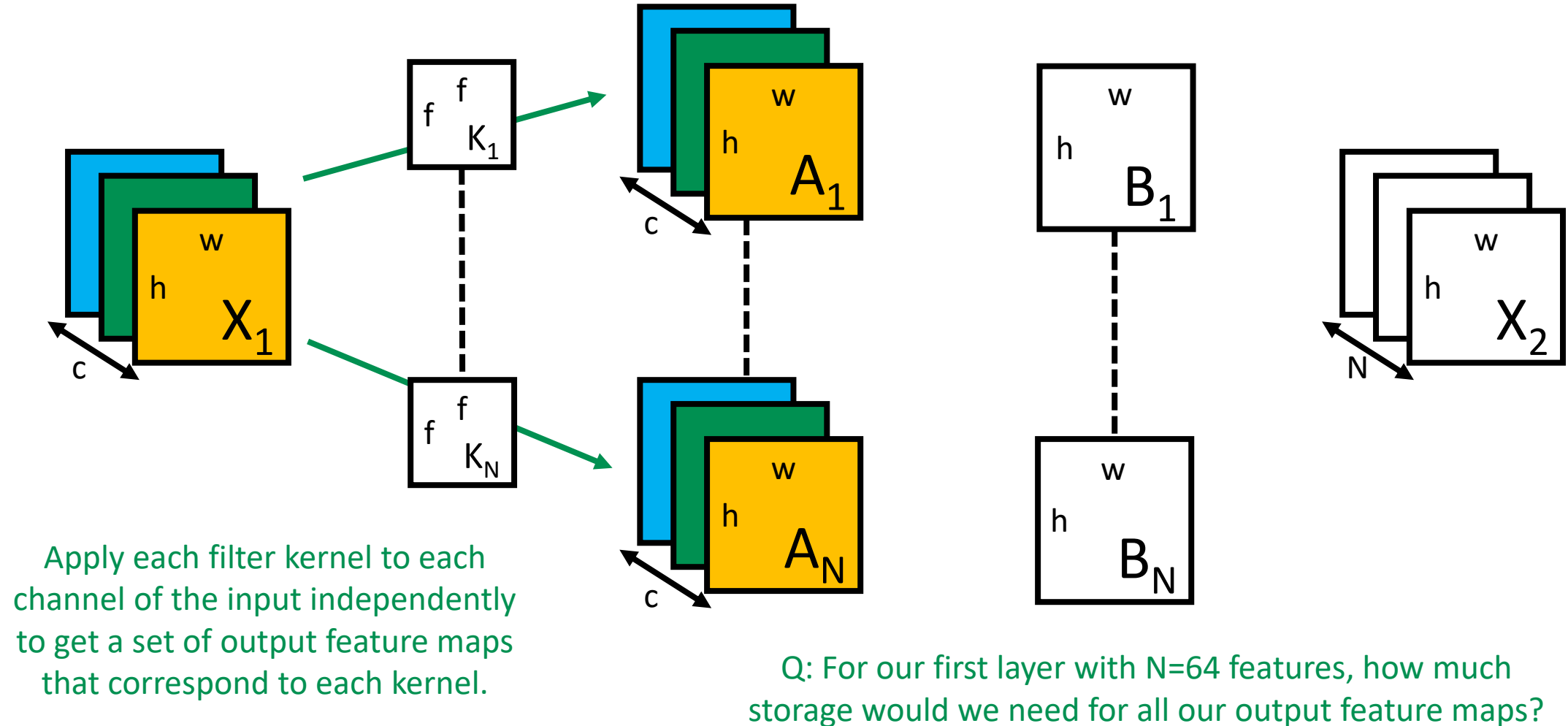
Operations in One Layer

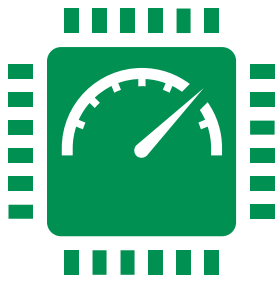


Layer	Kernel (f)	Features (N)	Activation (g)
0*			Bicubic*
1	9 x 9	64	ReLU
2	1 x 1	32	ReLU
3	5 x 5	1	None

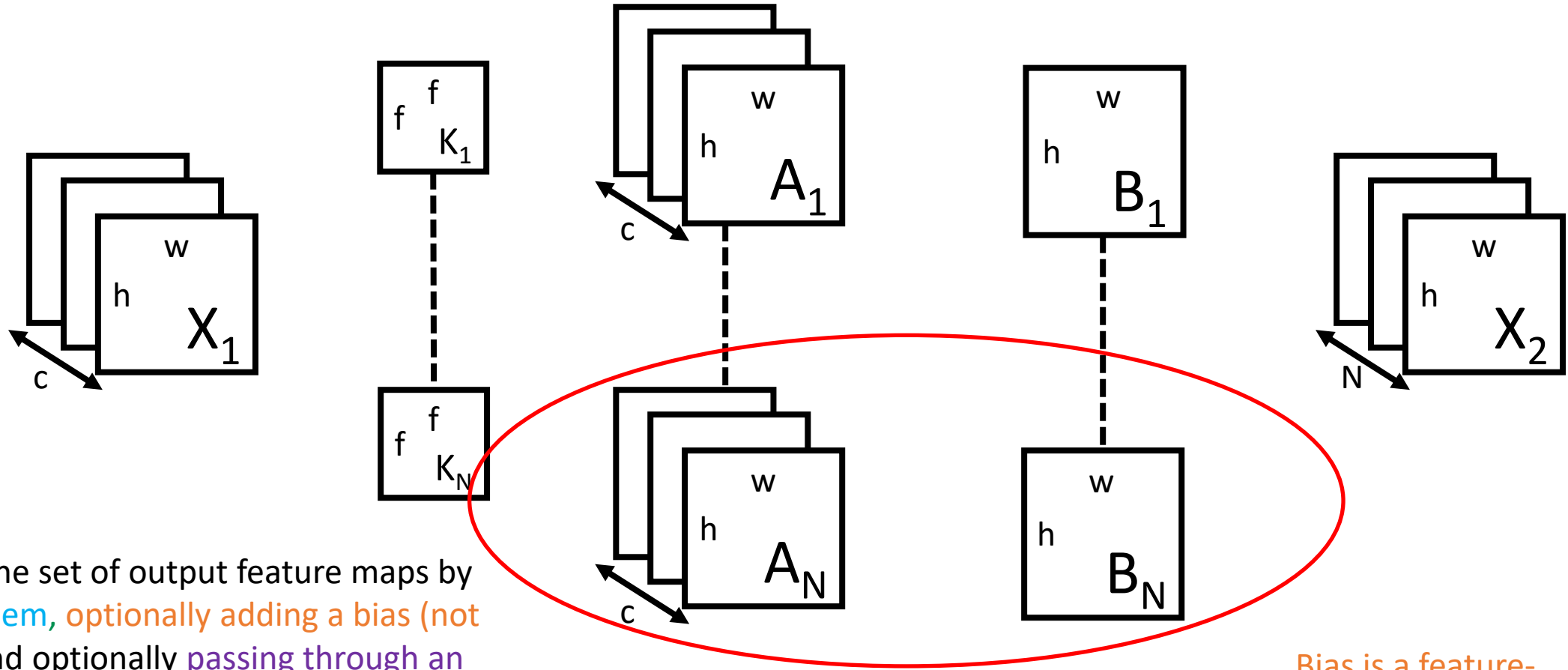


Operations in One Layer





Operations in One Layer

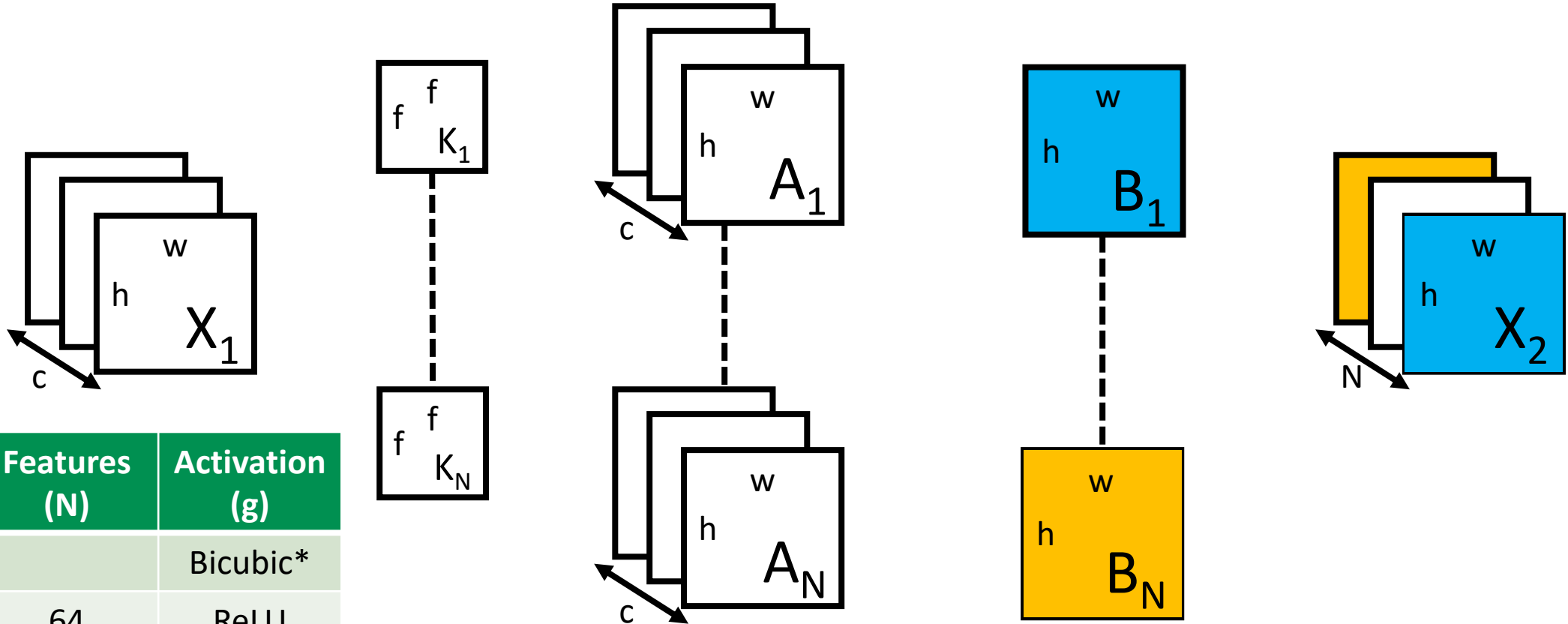
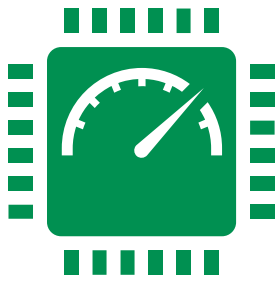


Combine one set of output feature maps by **summing them**, **optionally adding a bias (not shown)**, and optionally **passing through an activation function**.

$$B_n = g \left(\left(\sum_c A_n \right) + b_n \right)$$

Bias is a feature-specific scalar added to every pixel

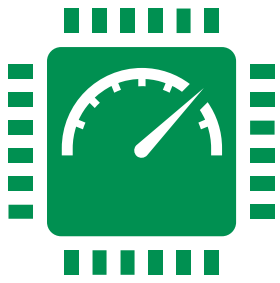
Operations in One Layer



Layer	Kernel (f)	Features (N)	Activation (g)
0*			Bicubic*
1	9 x 9	64	ReLU
2	1 x 1	32	ReLU
3	5 x 5	1	None

Stack the activations together. This is the output of this layer and the input to the next.

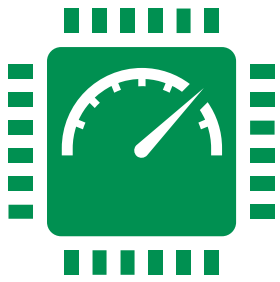
CNN Optimizations



1. Tiled convolution
2. Model quantization
3. Dataflow considerations

Tiled Convolution

Convolutional Layer



The basic convolutional layer implements a 2D discrete convolution between a filter kernel and an input feature map

Filter kernel
 $f_i \times f_i$

0	1	2
2	2	0
0	1	2

Input feature map
 $h_{i-1} \times w_{i-1}$

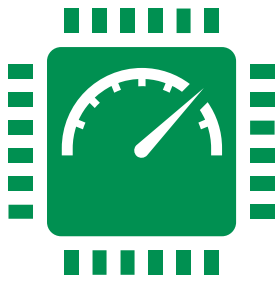
3	3	2	1	0
0	0	1	3	1
3	1	2	2	3
2	0	0	2	2
2	0	0	0	1

Output feature map
 $h_i \times w_i$

12.0	12.0	17.0
10.0	17.0	19.0
9.0	6.0	14.0

The diagram illustrates the convolution operation: a 3x3 filter kernel is convolved with a 5x5 input feature map to produce a 3x3 output feature map. The filter kernel is a 3x3 grid of integers. The input feature map is a 5x5 grid of integers. The output feature map is a 3x3 grid of floating-point numbers. The operation is represented by an asterisk (*) between the filter kernel and the input feature map, followed by an equals sign (=) and the output feature map.

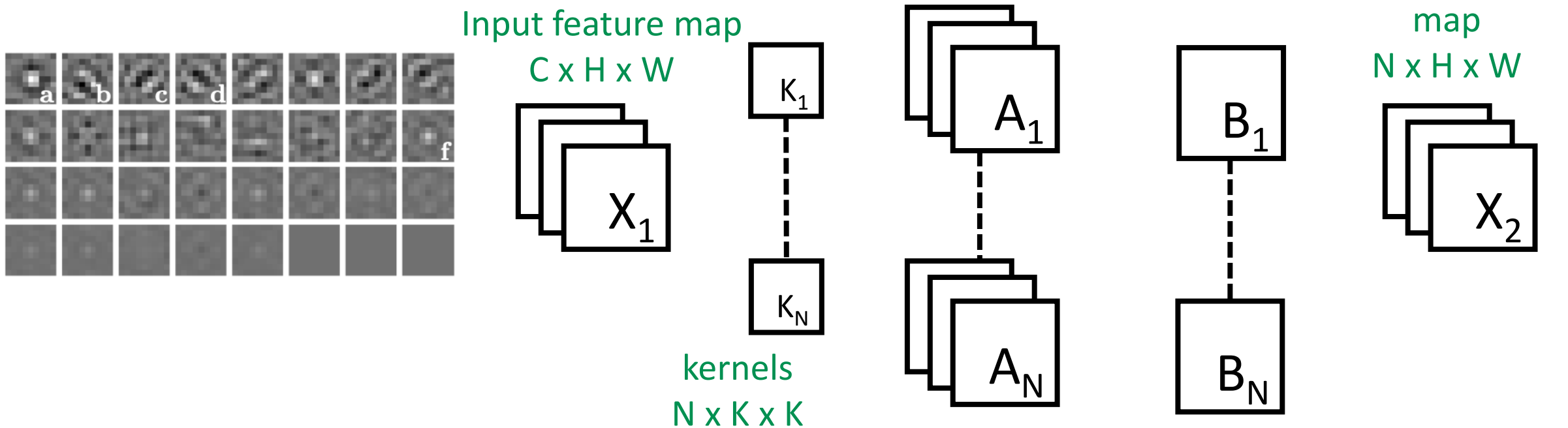
[3] Vincent Dumoulin, Francesco Visin - [A guide to convolution arithmetic for deep learning](#)

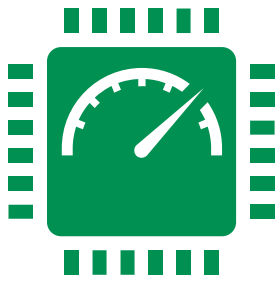


3D Feature Maps

Feature maps typically three dimensional

- Multichannel inputs (RGB or Y'CbCr, etc.)
- Multiple output features at each layer





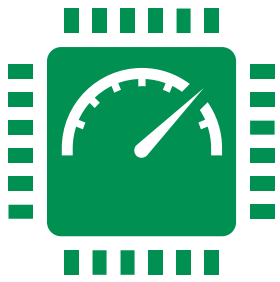
SRCNN Complexity

```
// Direct convolution
for (n = 0; n < N; n++) { // output feature
  for (h = 0; h < H; h++) { // output height
    for (w = 0; w < W; w++) { // output width
      for (c = 0; c < C; c++) { // input feature
        for (kh = 0; kh < K; kh++) { // kernel height
          for (kw = 0; kw < K; kw++) { // kernel width
            out[n][h][w] += W[c][kh][kw] * in[c][h+kh][w+kw];
          }
        }
      }
      out[n][w][h] = f(out[n][w][h] + b[n]); // bias & activation
    }
  }
}
```

What is complexity (in # of MACs) of SRCNN conv1 layer for an input image of dimensions 256 x 256?

Layer	Kernel	Features	Activation
0		1	
1	9 x 9	64	ReLU
2	1 x 1	32	ReLU
3	5 x 5	1	None

XCK26 SOM	
DSPs	1248
Logic cells	256k



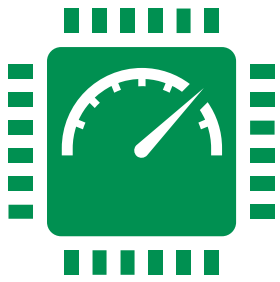
SRCNN Complexity

```
// Direct convolution
for (n = 0; n < N; n++) { // output feature
  for (h = 0; h < H; h++) { // output height
    for (w = 0; w < W; w++) { // output width
      for (c = 0; c < C; c++) { // input feature
        for (kh = 0; kh < K; kh++) { // kernel height
          for (kw = 0; kw < K; kw++) { // kernel width
            out[n][h][w] += W[c][kh][kw] * in[c][h+kh][w+kw];
          }
        }
        out[n][w][h] = f(out[n][w][h] + b[n]); // bias & activation
      }
    }
  }
}
```

How much RAM (in bits) is required to store the output feature map assuming single precision floating-point values?

Layer	Kernel	Features	Activation
0		1	
1	9 x 9	64	ReLU
2	1 x 1	32	ReLU
3	5 x 5	1	None

XCK26 SOM	
LUTRAM	3.2 Mb
BRAM	5.1 Mb
URAM	18 Mb



Block Matrix Multiplication

Blocking or *tiling* breaks computation of larger matrix down into computation of submatrices

- Reduce memory storage requirements and stream data to core as needed

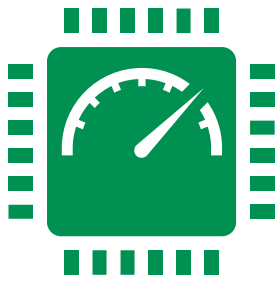
A_{11}	A_{12}	A_{13}	A_{14}
A_{21}	A_{22}	A_{23}	A_{24}
A_{31}	A_{32}	A_{33}	A_{34}
A_{41}	A_{42}	A_{43}	A_{44}

 $\times$

B_{11}	B_{12}	B_{13}	B_{14}
B_{21}	B_{22}	B_{23}	B_{24}
B_{31}	B_{32}	B_{33}	B_{34}
B_{41}	B_{42}	B_{43}	B_{44}

 $=$

AB_{11}	AB_{12}	AB_{13}	AB_{14}
AB_{21}	AB_{22}	AB_{23}	AB_{24}
AB_{31}	AB_{32}	AB_{33}	AB_{34}
AB_{41}	AB_{42}	AB_{43}	AB_{44}



Tiled Convolution

Tiled convolution allows similar parameterization of unrolling, memory requirements, and overlapping of data streaming from DRAM [1]

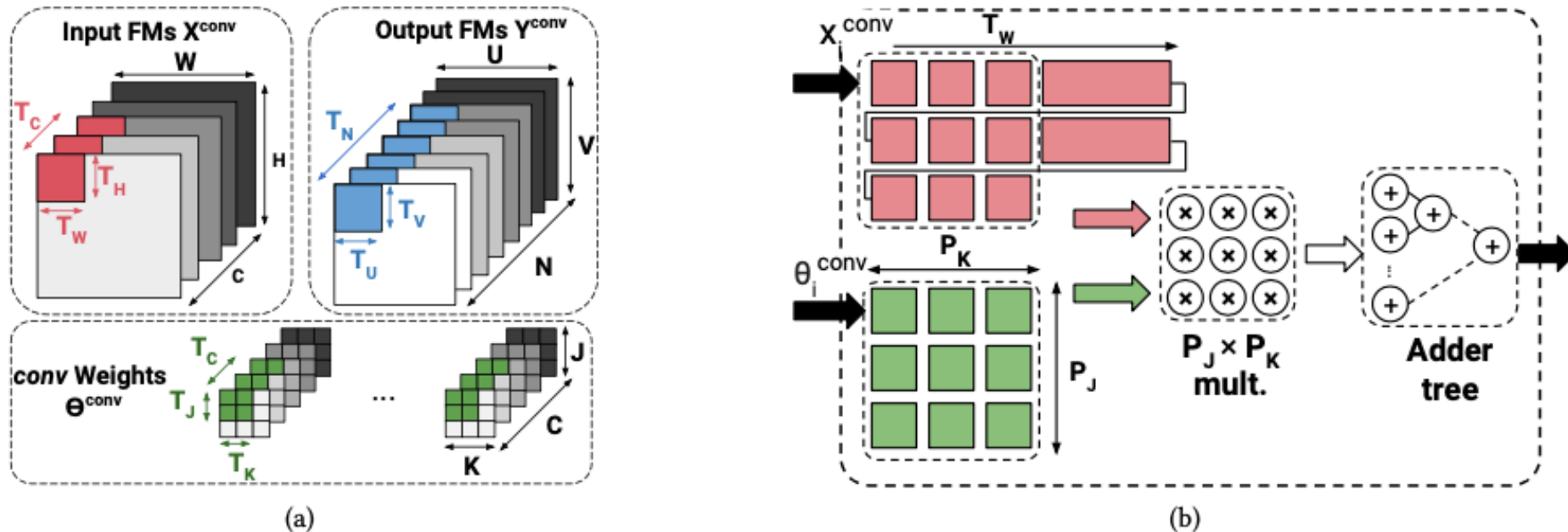


Figure 5: Loop tiling and unrolling

Abdelouahab, Kamel, et al. "Accelerating CNN inference on FPGAs: A survey."

// Tiled-convolution

```
for (n = 0; n < N; n += Tn) { // output feature
for (h = 0; h < H; h += Th) { // output height
for (w = 0; w < W; w += Tw) { // output width
for (c = 0; c < C; c += Tc) { // input feature
    // load input feature tile [c:c+Tc][h:h+Th][w:w+Tw]
    // load weight tile [n:n+Tn][c:c+Tc][0:K][0:K]
    // load output feature tile [n:n+Tn][h:h+Th][w:w+Tw]
```

```
    // tile calculation
```

```
    for (tn = 0; tn < Tn; tn++) { // output tile feature
    for (th = 0; th < Th; th++) { // output tile height
    for (tw = 0; tw < Tw; tw++) { // output tile width
    for (tc = 0; tc < Tc; tc++) { // input tile feature
        for (kh = 0; kh < K; kh++) { // kernel height
        for (kw = 0; kw < K; kw++) { // kernel width
            out[tn][th][tw] += W[tn][tc][kh][kw] * in[tc][th+kh][tw+kw];
        }
    }
    }
    }
}
```

```
    // store output feature tile [n:n+Tn][h:h+Th][w:w+Tw]
}
```

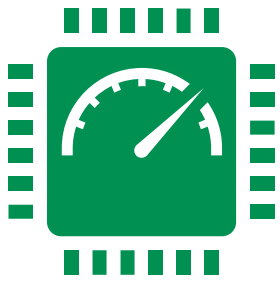
Tile across output features, width, height, input features. Design choice of:

- Loop schedule (order)
- Tile size (Tn, Th, Tw, Tc)

Input feature, output feature, and weight tiles must be loaded from DRAM into local memory

Loop unrolling and pipelining of tile calculation.

Updated output feature tile must be saved back to DRAM



Tile Order – Output Stationary

- Loop schedule (or order) will impact memory access pattern to load/store input feature map, weights, and output feature map from DRAM

// Output stationary

```
for (n = 0; n < N; n += Tn) { // output feature
for (h = 0; h < H; h += Th) { // output height
for (w = 0; w < W; w += Tw) { // output width
```

```
    // initialize output feature tile to zero
```

```
    for (c = 0; c < C; c += Tc) { // input feature
        // load input feature tile [c:c+Tc][h:h+Th][w:w+Tw]
        // load weight tile [n:n+Tn][c:c+Tc][0:K][0:K]
```

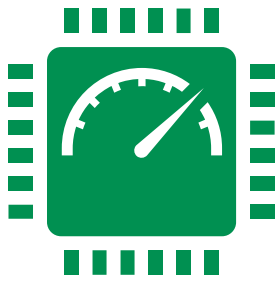
```
        // tile calculation
```

```
    }
    // store output feature tile [n:n+Tn][h:h+Th][w:w+Tw]
```

```
    }
```

Output stationary

Output tile initialized to zero, fully computed, and then written once to DRAM



Tile Order – Input Stationary

- Loop schedule (or order) will impact memory access pattern to load/store input feature map, weights, and output feature map from DRAM

```
// Input stationary
```

```
for (c = 0; c < C; c += Tc) { // input feature  
  for (h = 0; h < H; h += Th) { // output height  
    for (w = 0; w < W; w += Tw) { // output width
```

```
      // load input feature tile [c:c+Tc][h:h+Th][w:w+Tw]
```

```
        for (n = 0; n < N; n += Tn) { // output feature  
          // load output feature tile [n:n+Tn][h:h+Th][w:w+Tw]  
          // load weight tile [n:n+Tn][c:c+Tc][0:K][0:K]
```

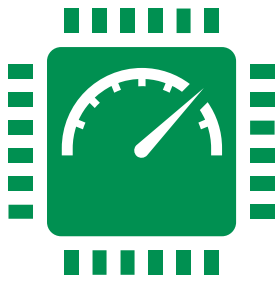
```
          // tile calculation
```

```
          // store output feature tile [n:n+Tn][h:h+Th][w:w+Tw]
```

```
        }  
      }  
    }  
  }  
}
```

Input stationary

Each input tile is loaded ONCE from DRAM, partial results for all affected output tiles computed and stored to DRAM



Tile Order – Weight Stationary

- Loop schedule (or order) will impact memory access pattern to load/store input feature map, weights, and output feature map from DRAM

// Weight stationary

for (n = 0; n < N; n += Tn) { // output feature

for (c = 0; c < C; n += Tc) { // input feature

// load weight tile [n:n+Tn][c:c+Tc][0:K][0:K]

for (h = 0; h < H; h += Th) { // output height

for (w = 0; w < W; w += Tw) { // output width

// load input feature tile [c:c+Tc][h:h+Th][w:w+Tw]

// load output feature tile [n:n+Tn][h:h+Th][w:w+Tw]

// tile calculation

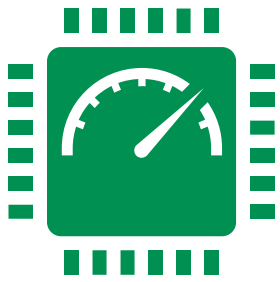
// store output feature tile [n:n+Tn][h:h+Th][w:w+Tw]

}}

}}

Weight stationary

Each weight tile is loaded ONCE from DRAM, partial results for all affected output tiles computed and stored to DRAM

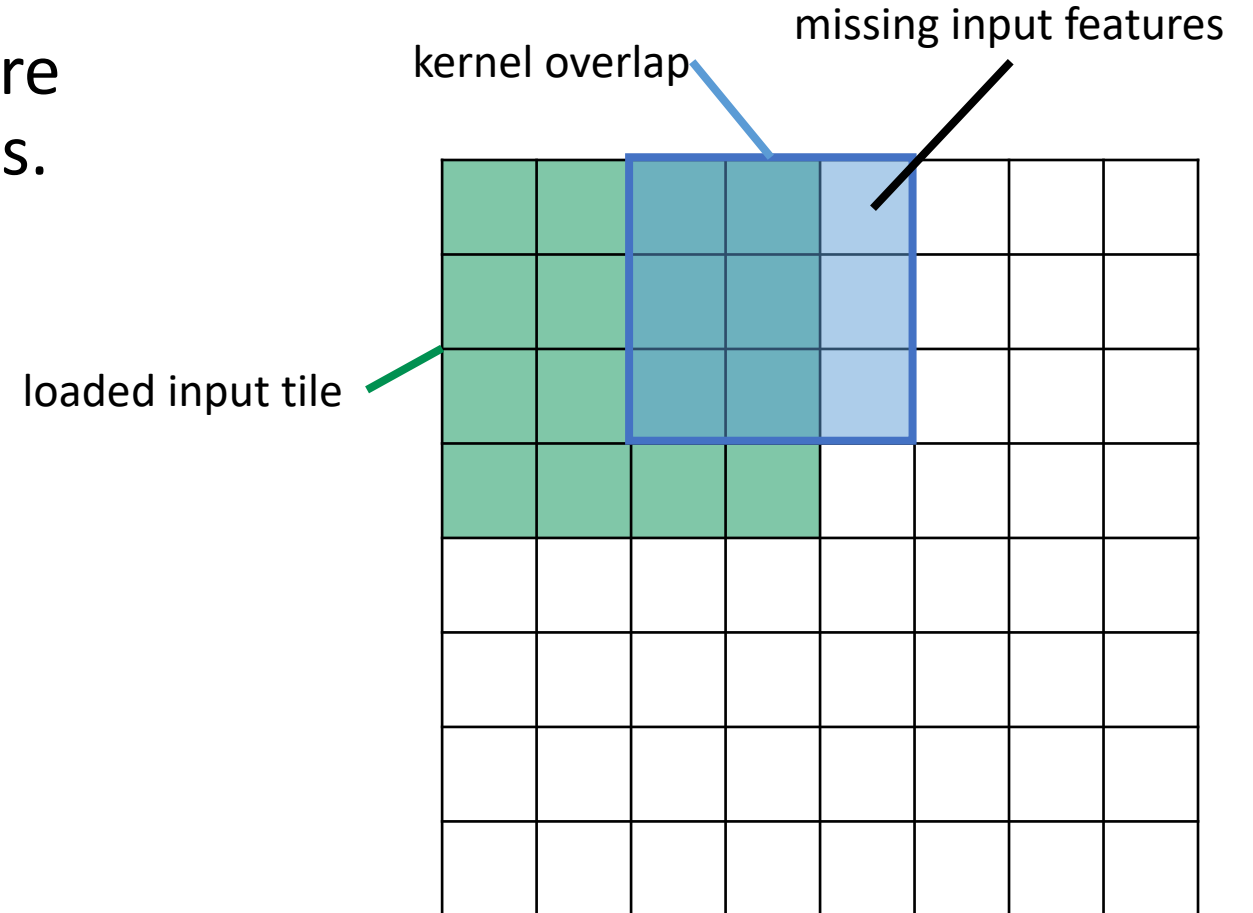


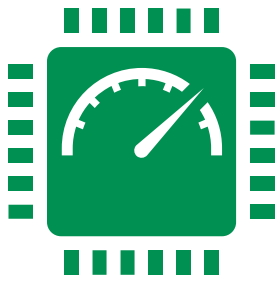
Tile Boundaries

Like padding in general case, care must be taken at tile boundaries.

Options include:

- Load additional input feature borders
- Store partial sums of output feature accumulations





Tile Calculation

■ Loop order

// Tile calculation

```
for (tn = 0; tn < Tn; tn++) { // output tile feature
for (th = 0; th < Th; th++) { // output tile height
for (tw = 0; tw < Tw; tw++) { // output tile width
for (tc = 0; tc < Tc; tc++) { // input tile feature
for (kh = 0; kh < K; kh++) { // kernel height
for (kw = 0; kw < K; kw++) { // kernel width
    out[tn][th][tw] += W[tn][tc][kh][kw] * in[tc][th+kh][tw+kw];
}}}}}
```

Tile calculation is your hardware accelerator

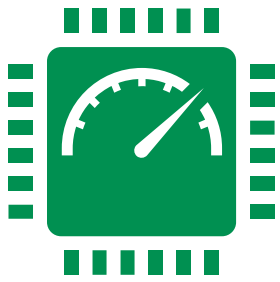
Apply pipelining and unrolling to loops

- Design choice of loop order

Input/Weight/Output tiles stored in on-chip memory (LUTRAM or BRAM)

- Array partitioning
- Local data re-use as possible

Model Quantization



Fixed-point Conversion

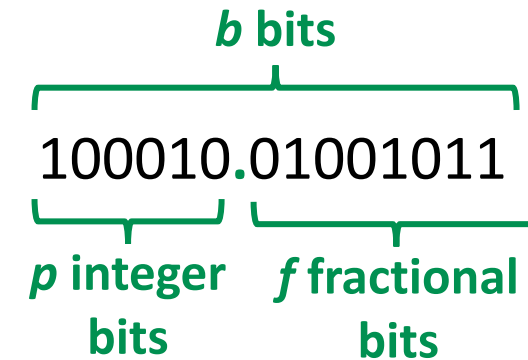
SRCNN algorithm and your golden reference implemented in floating-point

- Floating-point arithmetic requires more resources than fixed-point

Converting to fixed-point can allow more

- At each layer choose must quantize output feature maps, weights, and biases
- Can tradeoff some reduction in quality, i.e., MSE, for gains in throughput, latency, or area.

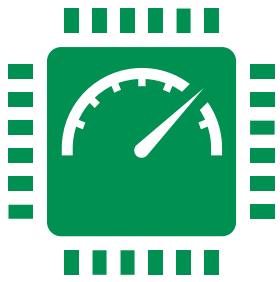
Fixed-point representation



For a fixed bit width, tradeoff between range for avoiding overflow and step size for reducing quantization errors

$$R = 2^{-f} 2^b$$

Fixed-point Conversion



Model conversion approach of [4]:

1. Apply floating point model to set of typical inputs and record all intermediate feature maps
2. Calculate statistics on weights, biases, and output feature maps of each layer
3. Choose a fixed-point bit width b and step size 2^{-f} for each item in (2)

See [4] for discussion on choosing (3) for different distributions

Lin et al., "Fixed point quantization of deep convolutional networks."

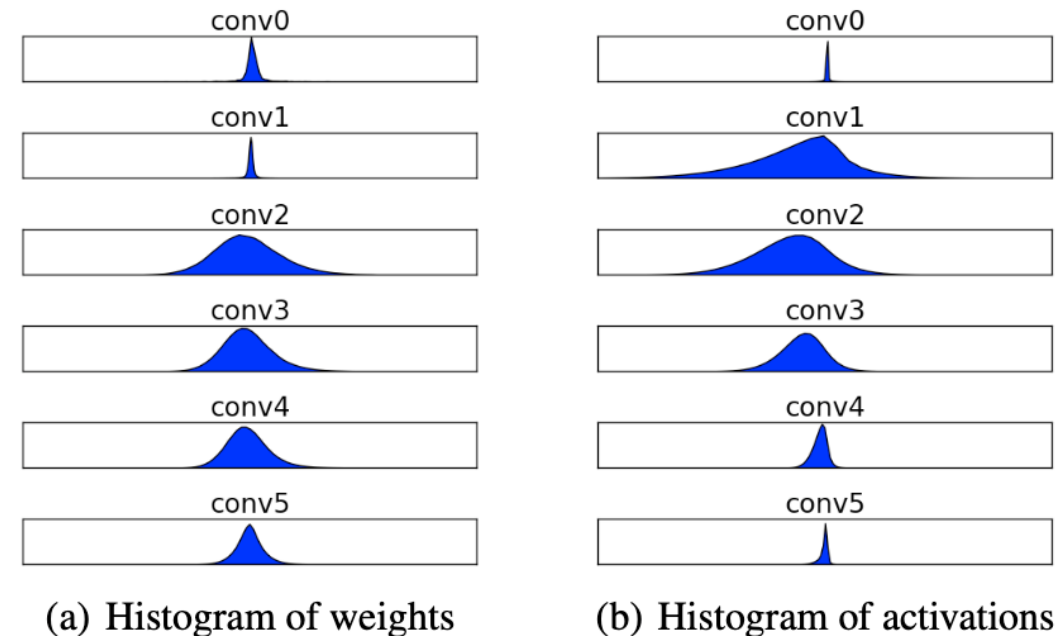
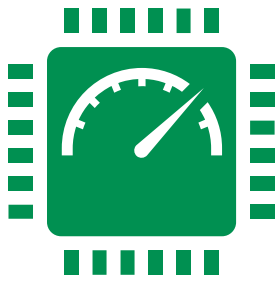


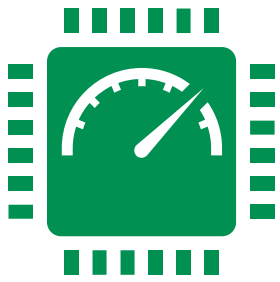
Figure 2. Distribution of weights & activations in a DCN design for CIFAR-10 benchmark.



CNN References

1. Zhang, Chen, et al. "Optimizing FPGA-based accelerator design for deep convolutional neural networks." *Proceedings of the 2015 ACM/SIGDA international symposium on field-programmable gate arrays*. 2015.
2. Abdelouahab, Kamel, et al. "Accelerating CNN inference on FPGAs: A survey." *arXiv preprint arXiv:1806.01683* (2018).
3. Williams, Samuel, Andrew Waterman, and David Patterson. "Roofline: an insightful visual performance model for multicore architectures." *Communications of the ACM* 52.4 (2009): 65-76.
4. Lin, Darryl, Sachin Talathi, and Sreekanth Annapureddy. "Fixed point quantization of deep convolutional networks." *International conference on machine learning*. PMLR, 2016.

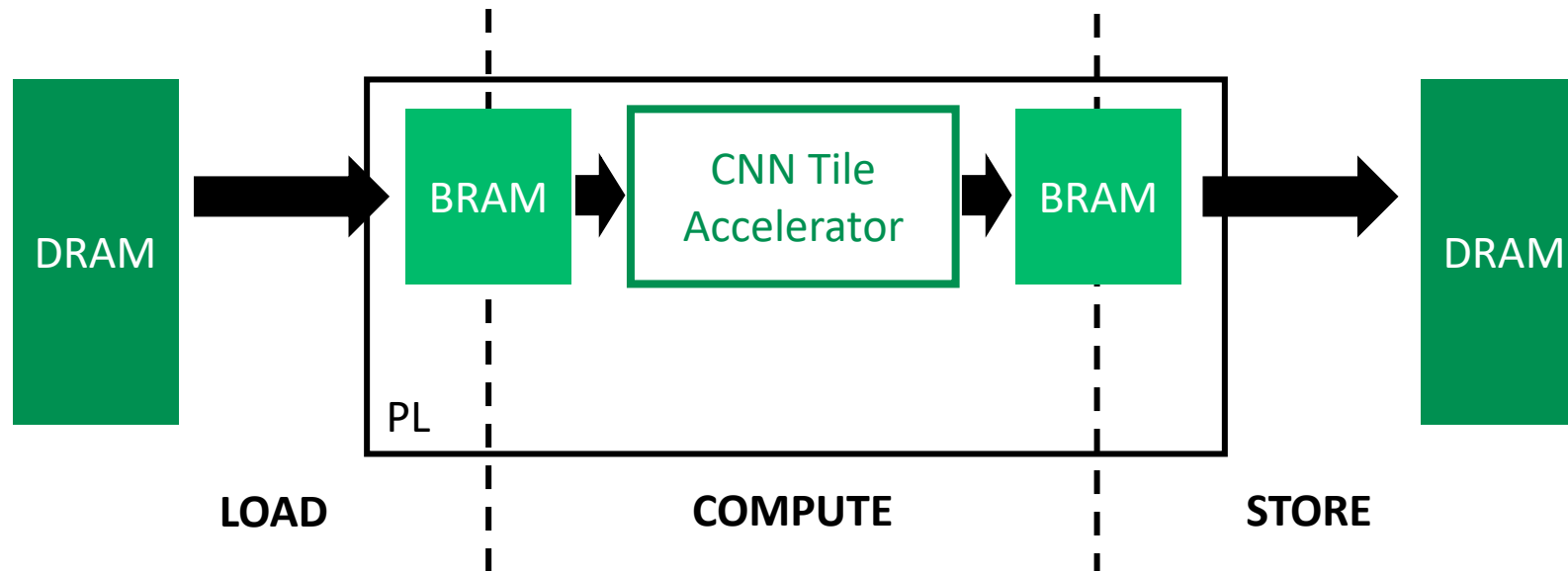
CNN Dataflow

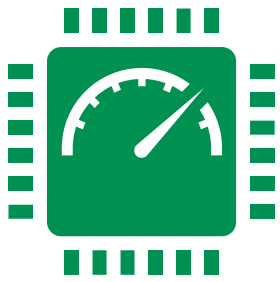


CNN Dataflow

Have seen that full feature maps cannot fit into on-chip memory

- DRAM access has high latency compared with accessing on-chip memory
- Must read/write feature map and weight tiles from DRAM to on-chip memory before computation
- Ideally, load/compute/write operations occur concurrently via PIPOs / FIFOs / Dataflow





HLS Interfaces

HLS synthesizes arguments of the top-level function to ports and interfaces to connect with external components

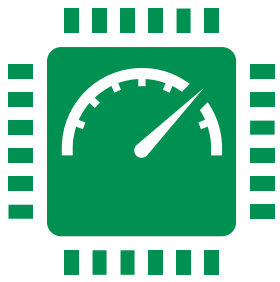
- IP interface defines how data flows into/out of IP, control of dataflow, and core execution control

For Vivado IP flow, default interfaces created are for integrating core with other IP

- Generally, will need to modify to make hardware software-controllable

Table: Interface Types

Paradigm	Description	Interface Types
Memory	Data is accessed by the kernel through memory such as DDR, HBM, PLRAM/BRAM/URAM	<code>ap_memory</code> , <code>bram</code> , AXI4 Memory Mapped (<code>m_axi</code>)
Stream	Data is streamed into the kernel from another streaming source, such as video processor or another kernel, and can also be streamed out of the kernel.	<code>ap_fifo</code> , AXI4-Stream (<code>axis</code>)
Register	Data is accessed by the kernel through register interfaces performed by register reads and writes.	<code>ap_none</code> , <code>ap_hs</code> , <code>ap_ack</code> , <code>ap_ovld</code> , <code>ap_vld</code> , and AXI4-Lite adapter (<code>s_axilite</code>).



Default Interfaces

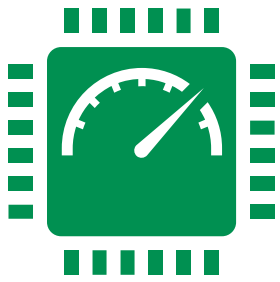
```
#include <hls_stream.h>

void my_ip(int  var0,
          int *var1,
          int  var2[100],
          hls::stream<int> &var3)
{
    *var1 = var0;

    for (int i=0; i < 100; i++)
        var3.write(var2[i]);
}
```

Table: Default Interfaces

C-Argument Type	Supported Paradigms	Default Paradigm	Default Interface Protocol		
			Input	Output	Inout
Scalar variable (pass by value)	Register	Register	ap_none	N/A	N/A
Array	Memory, Stream	Memory	ap_memory	ap_memory	ap_memory
Pointer	Memory, Stream, Register	Register	ap_none	ap_vld	ap_ovld
Reference	Register	Register	ap_none	ap_vld	ap_vld
<code>hls::stream</code>	Stream	Stream	ap_fifo	ap_fifo	N/A

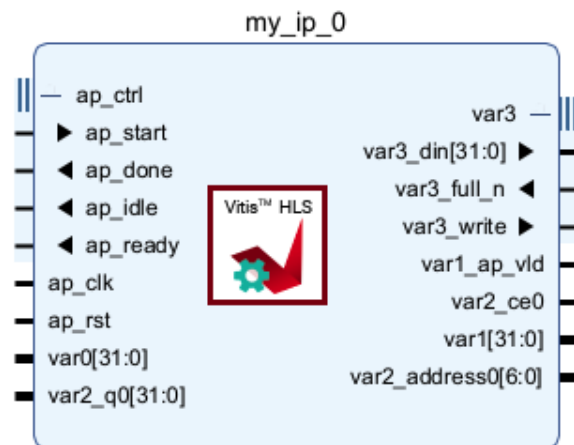


Default Interfaces

```
#include <hls_stream.h>

void my_ip(int  var0,
           int *var1,
           int  var2[100],
           hls::stream<int> &var3)
{
    *var1 = var0;

    for (int i=0; i < 100; i++)
        var3.write(var2[i]);
}
```



My_ip (Pre-Production)

▼ HW Interfaces

▼ AP_FIFO

Interface	Direction	Data Width
var3	out	32

▼ AP_MEMORY

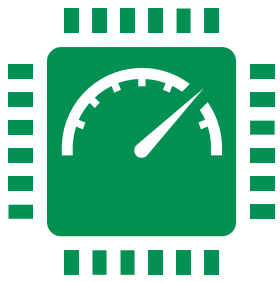
Port	Direction	Bitwidth
var2_address0	out	7
var2_q0	in	32

▼ Other Ports

Port	Mode	Direction	Bitwidth
var0	ap_none	in	32
var1	ap_vld	out	32

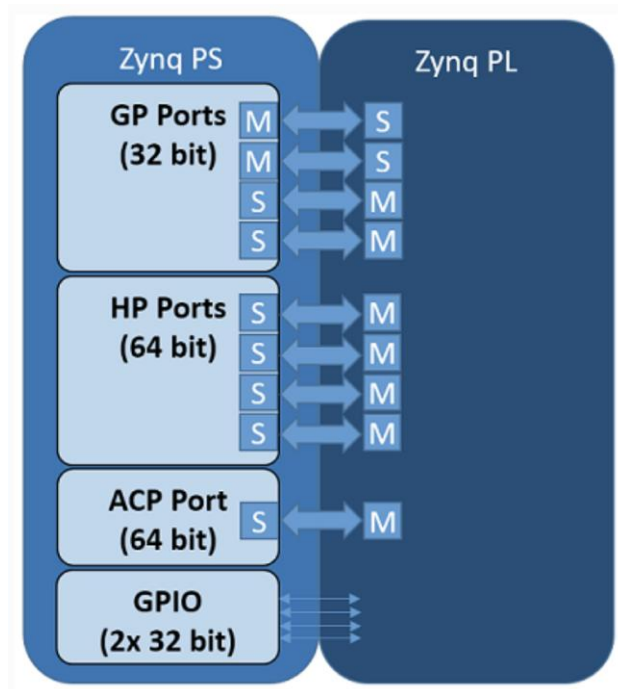
▼ TOP LEVEL CONTROL

Interface	Type	Ports
ap_clk	clock	ap_clk
ap_rst	reset	ap_rst
ap_ctrl	ap_ctrl_hs	ap_done ap_idle ap_ready ap_start

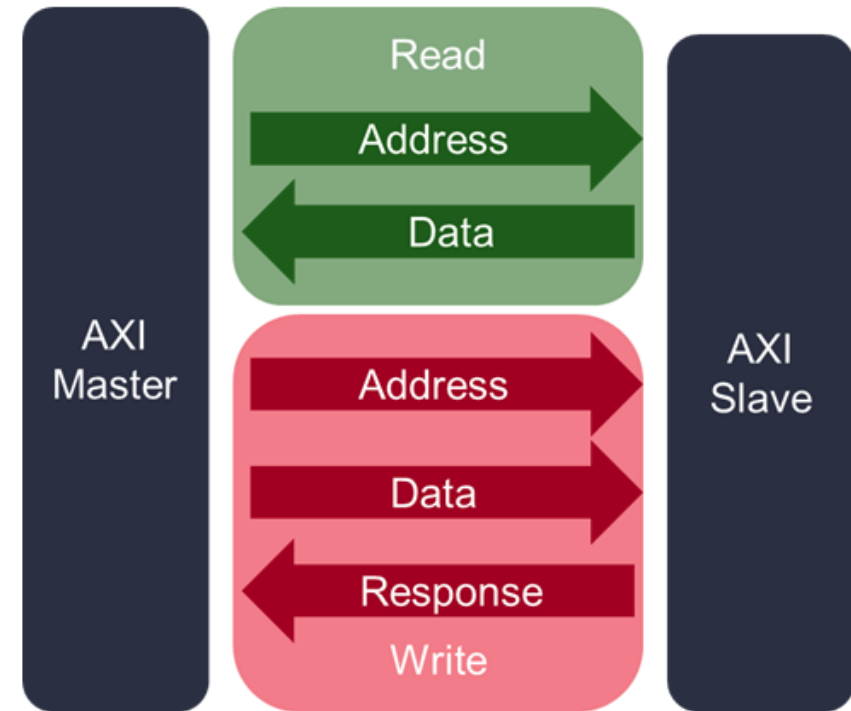


PS/PL Interface

We want our hardware cores to be controllable from software and to be able to directly access global memory (DRAM).

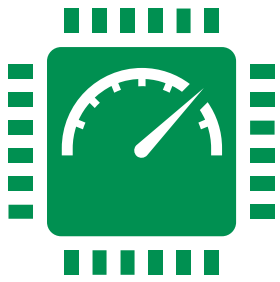


https://pynq.readthedocs.io/en/latest/overlay_design_methodology/pspl_interface.html



https://adaptivesupport.amd.com/s/article/1053914?language=en_US

AXI Bus



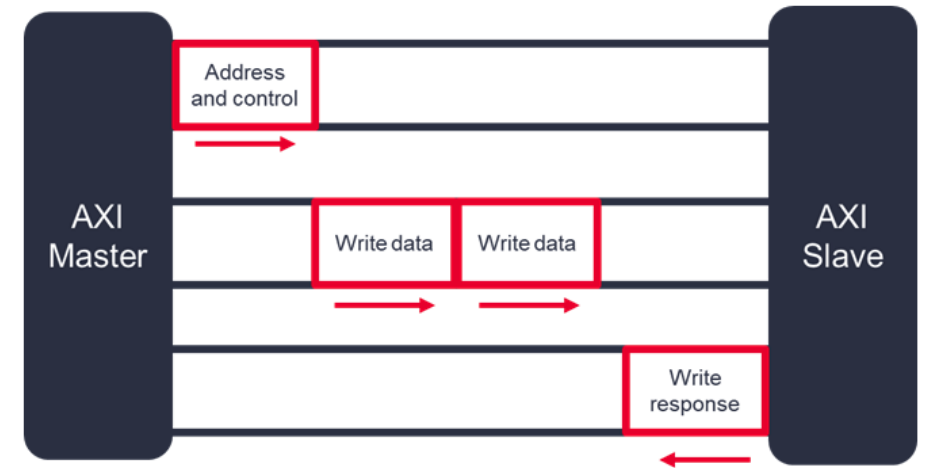
Advanced eXtensible Interface (AXI) is an ARM bus protocol for on-chip communications

- Between PS and PL
- Between PL and DRAM

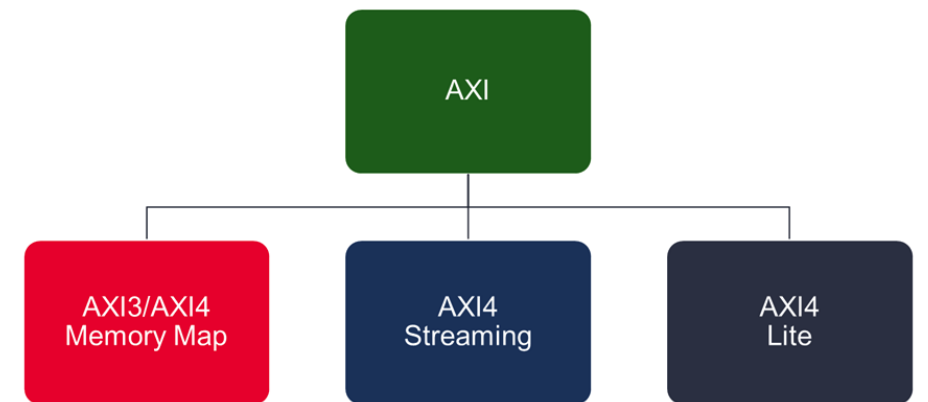
Wide bus width (up to 512 bits) with possibility of burst transfers

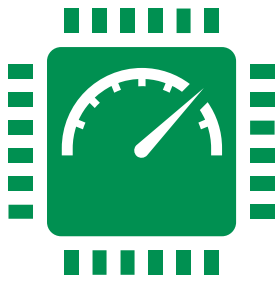
Three distinct modes for HLS: Memory Mapped, Streaming, and Lite

Can use `#pragma HLS INTERFACE` to have Vitis create AXI interfaces for your



https://adaptivesupport.amd.com/s/article/1053914?language=en_US





HLS Interface

```
void my_ip2(int var0,  
           int *var1,  
           int var2[100],  
           hls::stream<int> &var3)  
{  
#pragma HLS interface s_axilite port=return  
#pragma HLS interface s_axilite port=var0  
#pragma HLS interface s_axilite port=var1  
#pragma HLS interface m_axi      port=var2 offset=slave depth=100  
#pragma HLS interface axis      port=var3
```

AXI4 Lite

- Read and write registers in IP, including control
- Connect to PS AXI master interface

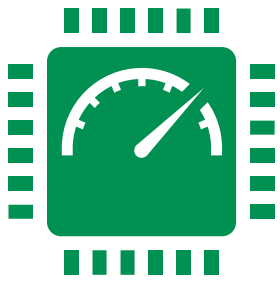
AXI4 Memory Mapped

- Read/write to global memory (DRAM)
- Ensure consecutive memory access to enable burst transfers of up to 4k bytes
- Connect to PS AXI slave interface

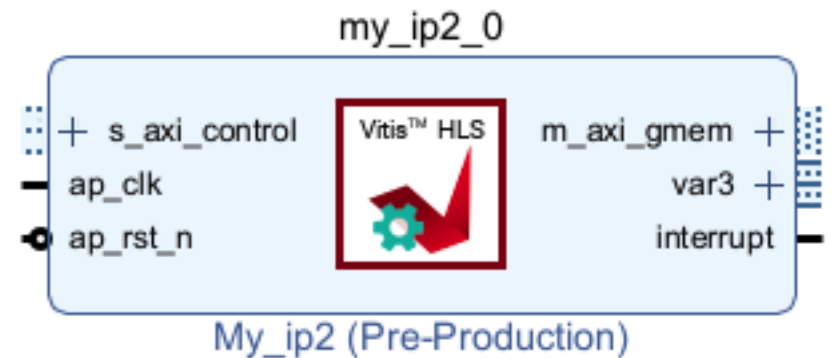
AXI4 Streaming

- Unidirectional streaming of unlimited burst size
- Connect to another streaming interface

HLS Interface



```
void my_ip2(int var0,  
           int *var1,  
           int var2[100],  
           hls::stream<int> &var3)  
{  
#pragma HLS interface s_axilite port=return  
#pragma HLS interface s_axilite port=var0  
#pragma HLS interface s_axilite port=var1  
#pragma HLS interface m_axi      port=var2 offset=slave depth=100  
#pragma HLS interface axis      port=var3
```



▼ HW Interfaces

▼ M_AXI

Interface	Data Width (SW->HW)	Address Width	Latency	Offset	Register	Max Widen Bitwidth	Max Read Burst Length	Max Write Burst Length	Num Read Outs
m_axi_gmem	32 -> 32	64	0	slave	0	0	16	16	

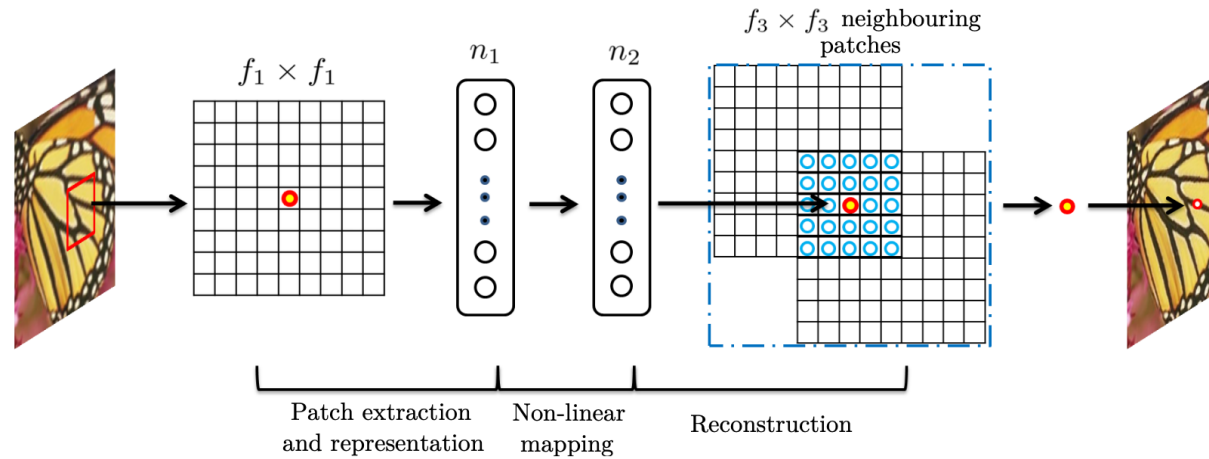
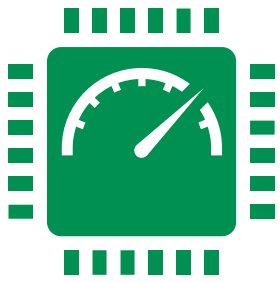
▼ S_AXILITE Registers

Interface	Register	Offset	Width	Access	Description	Bit Fields
s_axi_control	CTRL	0x00	32	RW	Control signals	0=AP_START 1=AP_DONE 2=AP_IDLE 3=AP_READY 7=AUTO_RESTART 9=INTERRUPT
s_axi_control	GIER	0x04	32	RW	Global Interrupt Enable Register	0=Enable
s_axi_control	IP_IER	0x08	32	RW	IP Interrupt Enable Register	0=CHAN0_INT_EN 1=CHAN1_INT_EN
s_axi_control	IP_ISR	0x0c	32	RW	IP Interrupt Status Register	0=CHAN0_INT_ST 1=CHAN1_INT_ST
s_axi_control	var0	0x10	32	W	Data signal of var0	
s_axi_control	var1	0x18	32	R	Data signal of var1	
s_axi_control	var1_ctrl	0x1c	32	R	Control signal of var1	0=var1_ap_vld
s_axi_control	var2_1	0x28	32	W	Data signal of var2	
s_axi_control	var2_2	0x2c	32	W	Data signal of var2	

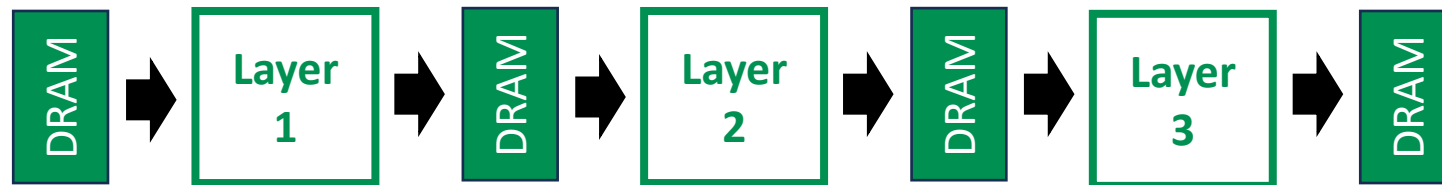
▼ AXIS

Interface	Direction	Register Mode	TDATA	TREADY	TVALID
var3	out	both	32	1	1

Multiple Layers



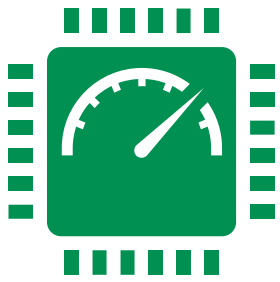
Option 1:



Option 2:



?



Dataflow Example

Overlapping of
load/compute/store
operations important for
maximizing throughput

```
#include <string.h>

#define M 1000
#define N 100

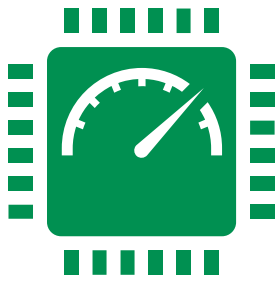
void dma_example(int a[M], int b[M])
{
    #pragma HLS INTERFACE mode=m_axi port=a offset=slave depth=M bundle=a
    #pragma HLS INTERFACE mode=m_axi port=b offset=slave depth=M bundle=b
    #pragma HLS INTERFACE mode=s_axilite port= return

    int a_local[N], b_local[N]; // local memory block

    for (int m = 0; m < M; m += N) {
        #pragma HLS DATAFLOW
        // load input block from DRAM - memcpy to create burst access
        memcpy(a_local, (const int *)&a[m], N*sizeof(int));

        // compute using local copies
        for (int n=0; n < N; n++)
            b_local[n] = a_local[n] + 1;

        // store output block in DRAM - memcpy to create burst access
        memcpy(&b[m], b_local, N*sizeof(int));
    }
}
```



Key Takeaways

1. Convolutional tiling will be required to fit SRCNN on Kria board
2. You have a design choices in tile size, tile order, and tile calculation pipelining/unrolling
3. Fixed-point conversion can allow more arithmetic units, and thus higher throughput, at a potential cost to output accuracy (MSE)
4. Global memory access (DRAM) likely to be a bottleneck in design, so important to consider dataflow and overlapping load/compute/store operations